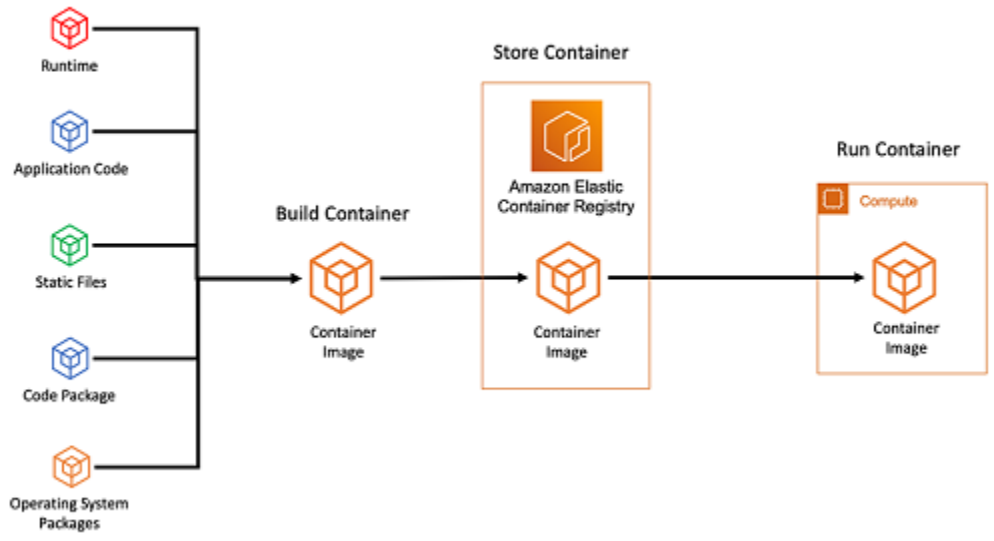
Best Practices - Running your application with Amazon ECS

Before you run an application using Amazon Elastic Container Service, make sure that you understand how the various aspects of your application work with features in Amazon ECS. This guide covers the main Amazon ECS resources types, what they're used for, and best practices for using each of these resource types.

Container image

A container image holds your application code and all the dependencies that your application code requires to run. Application dependencies include the source code packages that your application code relies on, a language runtime for interpreted languages, and binary packages that your dynamically linked code relies on.



Container images go through a three-step process.

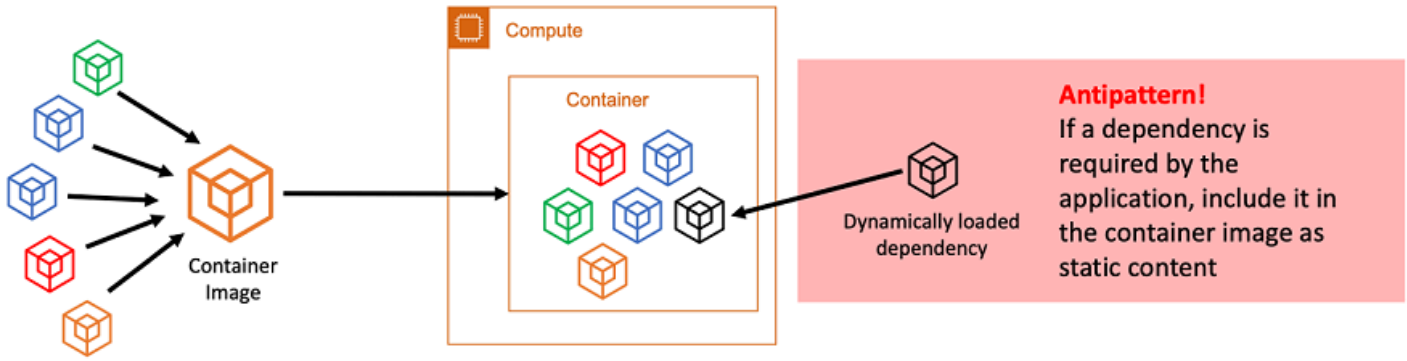
1. **Build** - Gather your application and all its dependencies into one container image.
2. **Store** - Upload the container image to a container registry.
3. **Run** - Download the container image on some compute, unpack it, and start the application.

When you create your own container image, keep in mind the best practices described in the following sections.

Make container images complete and static

Ideally, a container image is intended to be a complete snapshot of everything that the application requires to function. With a complete container image, you can run an application by downloading one container image from one place. You don't need to download several separate pieces from different locations. Therefore, as a best practice, store all application dependencies as static files inside the container image.

At the same time, don't dynamically download libraries, dependencies, or critical data during application startup. Instead, include these things as static files in the container image. Later on, if you want to change something in the container image, build a new container image with the changes applied to it.



There are a few reasons why we recommend this best practice.

- Including all dependencies as static files in the container image reduces the number of potentially breaking events that can happen during deployments. As you scale out to tens, hundreds, or even thousands of copies of your container, downloading a single container image rather than downloading from two or three different places simplifies your workload by limiting potential breaking points. For example, assume that you're deploying 100 copies of your application, and each copy of the application has to download pieces from three different sources. There are 300 downloads that can fail. If you're downloading a container image, there's only 100 dependencies that can break.
- Container image downloads are optimized for downloading the application dependencies in parallel. By default, a container image is made up of layers that can be downloaded and unpacked in parallel. This means that a container image download can get all of your

dependencies onto your compute faster than a hand coded script that downloads each dependency in a series.

- By keeping all your dependencies inside of the image, your deployments are more reliable and reproducible. If you change a dynamically loaded dependency, it might break the application inside the container image. However, if the container is truly standalone, you can always redeploy it, even in the future. This is because it already has the right versions and right dependencies inside of it.

Maintain fast container launch times by keeping container images as small as possible

Complete containers hold everything that's needed to run your application, but they don't need to include your build tools. Consider this example. Assume that you're building a container for a Node.js application. You must have the NPM package manager to download packages for your application. However, you no longer need NPM itself when the application runs. You can use a multistage Docker build to solve this.

The following is an example of what such a multistage `Dockerfile` might look like for a Node.js application that has dependencies in NPM.

```
FROM node:14 AS build
WORKDIR /srv
ADD package.json .
RUN npm install

FROM node:14-slim
COPY --from=build /srv .
ADD . .
EXPOSE 3000
CMD ["node", "index.js"]
```

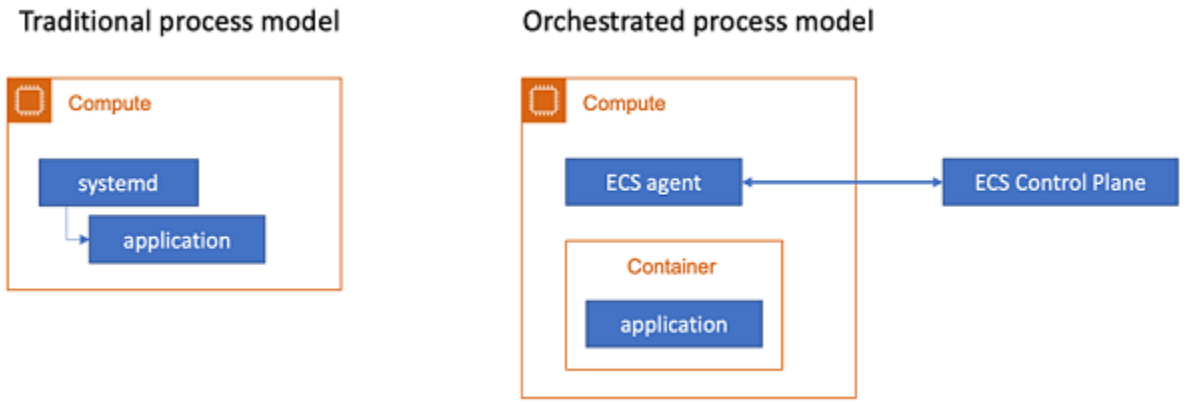
The first stage uses a full Node.js environment that has NPM, and a compiler for building native code bindings for packages. The second stage includes nothing but the Node.js runtime. It can copy the downloaded NPM packages out of the first stage. The final product is a minimal image that has the Node.js runtime, the NPM packages, and the application code. It doesn't include the full NPM build toolchain.

Keep your container images as small as possible and use shared layers. For example, if you have multiple applications that use the same data set, you can create a shared base image that has that data set. Then, build two different image variants off of the same shared base image. This allows the container image layer with the dataset to be downloaded one time, rather than twice.

The main benefit of using smaller container images is that these images can be downloaded onto compute hardware faster. This allows your application to scale out faster and quickly recover from unexpected crashes or restarts.

Only run a single application process with a container image

In a traditional virtual machine environment, it's typical to run a high-level daemon like `systemd` as the root process. This daemon is then responsible for starting your application process, and restarting the application process if it crashes. We don't recommend this when using containers. Instead, only run a single application process with a container.



If the application process crashes or ends, the container also ends. If the application must be restarted on crash, let Amazon ECS manage the application restart externally. The Amazon ECS agent reports to the Amazon ECS control plane that the application container crashed. Then, the control plane determines whether to launch a replacement container, and if so where to launch it. The replacement container may be placed onto the same host, or onto a different host in the cluster.

Treat containers as ephemeral resources. They only last for the lifetime of the main application process. Don't keep restarting application processes inside of a container, to try to keep the container up and running. Let Amazon ECS replace containers as needed.

This best practice has two key benefits.

- It mitigates scenarios where an application crashed because of a mutation to the local container filesystem. Instead of reusing the same mutated container environment, the orchestrator launches a new container based off the original container image. This means that you can be confident that the replacement container is running a clean, baseline environment.
- Crashed processes are replaced through a centralized decision making process in the Amazon ECS control plane. The control plane makes smarter choices about where to launch the replacement process. For example, the control plane can attempt to launch the replacement onto different hardware in a different Availability Zone. This makes the overall deployment more resilient than if each individual compute instance attempts to relaunch its own processes locally.

Handle `SIGTERM` within the application

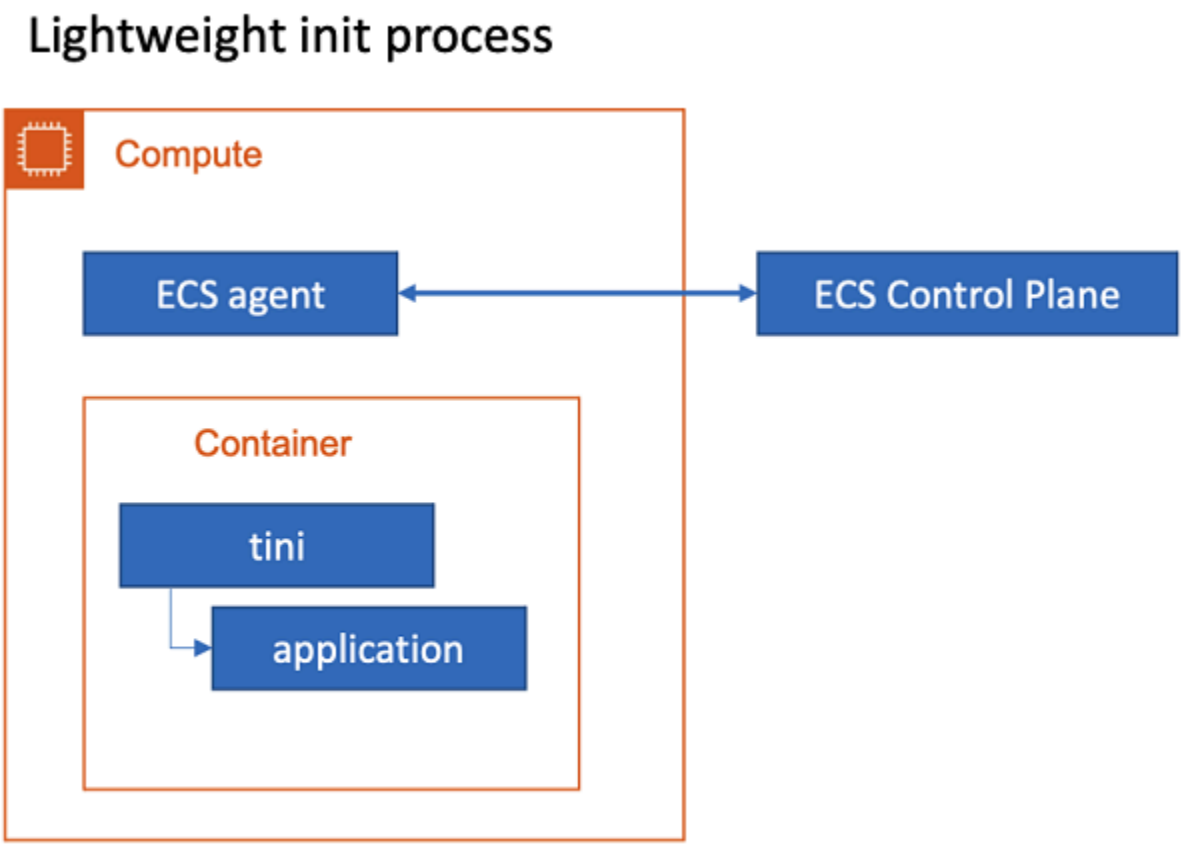
When you're following the guidance of the previous section, you're allowing Amazon ECS to replace tasks elsewhere in the cluster, rather than restart the crashing application. There are other times when a task may be stopped that are outside the application's control. Tasks may be stopped due to application errors, health check failures, completion of business workflows or even manual termination by a user.

When a task is stopped by ECS, ECS follows the steps and configuration shown in SIGTERM responsiveness.

To prepare your application, you need to identify how long it takes your application to complete its work, and ensure that your applications handles the `SIGTERM` signal. Within the application's signal handling, you need to stop the application from taking new work and complete the work that is in-progress, or save unfinished work to storage outside of the task if it would take too long to complete.

After sending the `SIGTERM` signal, Amazon ECS will wait for the time specified in the `StopTimeout` in the task definition. Then, the `SIGKILL` signal will be sent. Set the `StopTimeout` long enough that your application completes the `SIGTERM` handler in all situations before the `SIGKILL` is sent.

For web applications, you also need to consider open connections that are idle. See the following page of this guide for more details Network Load Balancer.



If you use an init process in your container, use a lightweight init process such as `tini`. This init process takes on the responsibility of reaping zombie processes if your application spawns worker processes. If your application doesn't handle the `SIGTERM` signal properly, `tini` can catch that signal for you and terminate your application process. However, if your application process crashes `tini` doesn't restart it. Instead `tini` exits, allowing the container to end and be replaced by container orchestration. For more information, see `tini` on GitHub.

Configure containerized applications to write logs to `stdout` and `stderr`

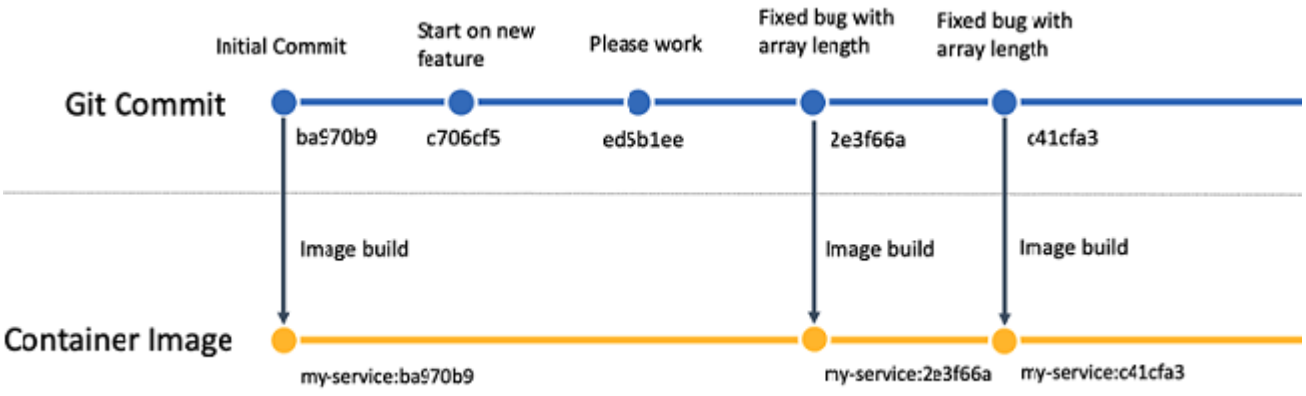
There are many different ways to do logging. For some application frameworks, it's common to use an application logging library that writes directly to disk files. It's also common to use one that streams logs directly to an ELK (OpenSearch, Logstash, Kibana) stack or a similar logging setup. However, we recommend that, when an application is containerized, you configure it to write application logs directly to the `stdout` and `stderr` streams.

Docker includes a variety of logging drivers that take the `stdout` and `stderr` log streams and handle them. You can choose to write the streams to `syslog`, to disk on the local instance that's running the container, or use a logging driver to send the logs to Fluentd, Splunk, CloudWatch, and other destinations. With Amazon ECS, you can choose to configure the FireLens logging driver. This driver can attach Amazon ECS metadata to logs, filter logs, and route logs to different destinations based on criteria such as HTTP status code. For more information about Docker logging drivers, see Configure logging drivers. For more information about FireLens, see Using FireLens.

When you decouple log handling from your application code, it gives you greater flexibility to adjust log handling at the infrastructure level. Assume that you want to switch from one logging system to another. You can do so by adjusting a few settings at the container orchestrator level, rather than having to change code in all your services, build a new container image, and deploy it.

Version container images using tags

Container images are stored in a container registry. Each image in a registry is identified by a tag. There's a tag called `latest`. This tag functions as a pointer to the latest version of the application container image, similar to the HEAD in a git repository. We recommend that you use the `latest` tag only for testing purposes. As a best practice, tag container images with a unique tag for each build. We recommend that you tag your images using the git SHA for the git commit that was used to build the image.



You don’t need to build a container image for every commit. However, we recommend that you build a new container image each time you release a particular code commit to the production environment. We also recommend that you tag the image with a tag that corresponds to the git commit of the code that's inside the image. If you tagged the image with the git commit, you can more quickly find which version of the code the image is running.

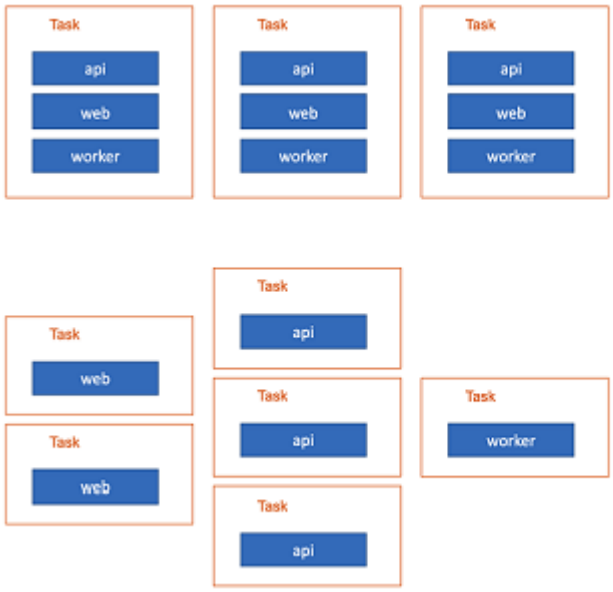
We also recommend that you turn on immutable image tags in Amazon Elastic Container Registry. With this setting, you can't change the container image that a tag points at. Instead Amazon ECR enforces that a new image must be uploaded to a new tag, rather than overwriting a pre-existing tag. For more information, see Image tag mutability in the *Amazon ECR User Guide*.

Task definition

The task definition is a document that describes what container images to run together, and what settings to use when running the container images. These settings include the amount of CPU and memory that the container needs. They also include any environment variables that are supplied to the container and any data volumes that are mounted to the container. Task definitions are grouped based on the dimensions of family and revision.

Use each task definition family for only one business purpose

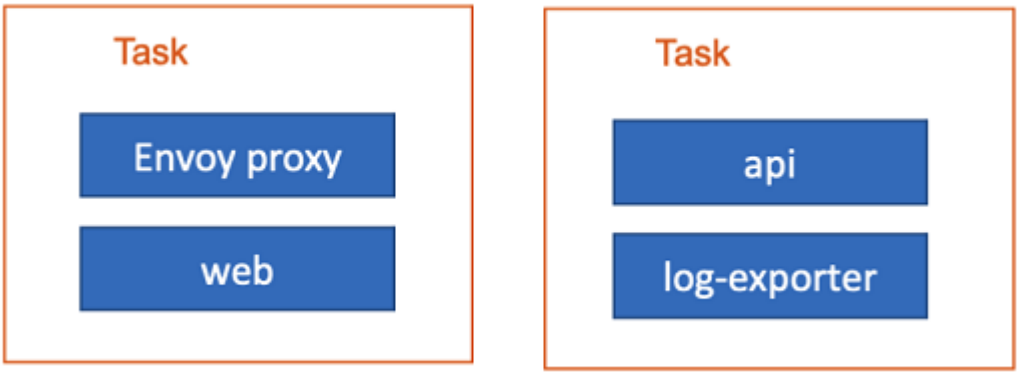
You can use an Amazon ECS task definition to specify multiple containers. All the containers that you specify are deployed along the same compute capacity. Don't use this feature to add multiple application containers to the same task definition because this prevents copies of each application scaling separately. For example, consider this situation. Assume that you have a web server container, an API container, and a worker service container. As a best practice, use a separate task definition family for each of these pieces of containerized code.



If you group multiple types of application container together in the same task definition, you can’t independently scale those containers. For example, it's unlikely that both a website and an API require scaling out at the same rate. As traffic increases, there will be a different number of web

containers required than API containers. If these two containers are being deployed in the same task definition, every task runs the same number of web containers and API containers.

We recommend that you scale each type of container independently based on demand.



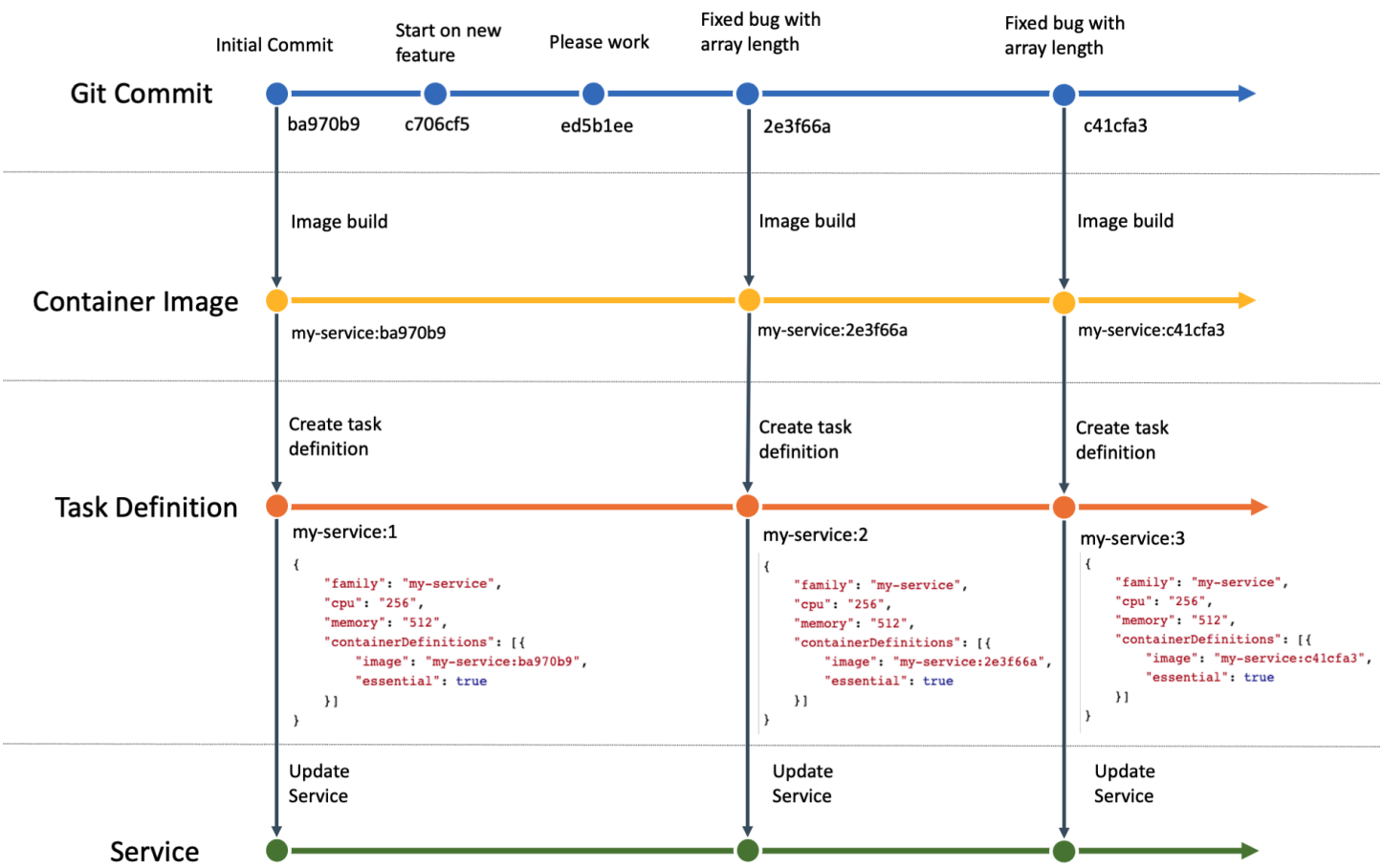
We don't recommend that you use multiple containers in a single task definition for grouping different types of application container. The purpose of having multiple containers in a single task definition is so that you can deploy sidecars, small addon containers that enhance a single type of container. A sidecar might help with logging and observability, traffic routing, or other addon features.

We recommend that you use sidecars to attach extra functionality, but that the task has a single business function.

Match each application version with a task definition revision within a task definition family

A task definition can be configured to point at any container image tag, including the “latest” tag. However, we don't recommend that you use the “latest” tag in your task definition. This is because “latest” tag functions as a mutable pointer, so the contents of the image it points at can change while Amazon ECS doesn't identify the modification.

Within a task definition family, consider each task definition revision as a point in time snapshot of the settings for a particular container image. This is similar to how the container is a snapshot of all the things that are needed to run a particular version of your application code.

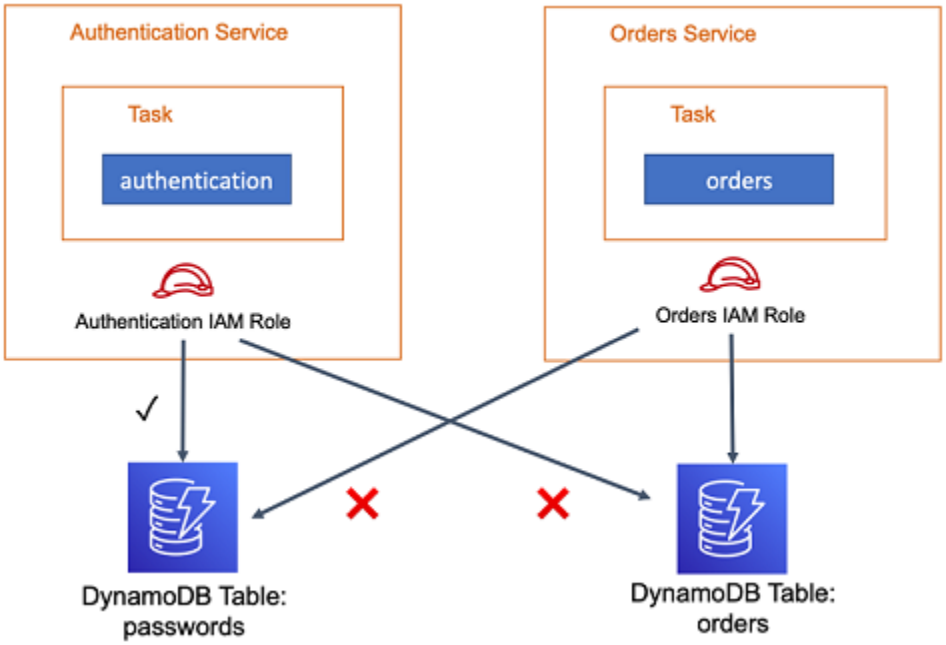


Make sure that there's a one-to-one mapping between a version of application code, a container image tag, and a task definition revision. A typical release process involves a git commit that gets turned into a container image that's tagged with the git commit SHA. Then, that container image tag gets its own Amazon ECS task definition revision. Last, the Amazon ECS service is updated to tell it to deploy the new task definition revision.

By using this approach, you can maintain consistency between settings and application code when rolling out new versions of your application. For example, assume that you make a new version of your application that uses a new environment variable. The new task definition that corresponds to that change also defines the value for the environment variable.

Use different IAM roles for each task definition family

You can define different IAM roles for different tasks in Amazon ECS. Use the task definition to specify an IAM role for that application. When the containers in that task definition are run, they can call AWS APIs based on the policies that are defined in the IAM role. For more information, see IAM roles for tasks.



Define each task definition with its own IAM role. This recommendation should be done in tandem with our recommendation for providing each business component its own task definition family. By implementing both of these best practices, you can limit how much access each service has to resources in your AWS account. For example, you can give your authentication service access to connect to your passwords database. At the same time, you can also ensure that only your order service has access to the credit card payment information.

Amazon ECS service

ECS uses the service resource to group, monitor, replace, and scale identical tasks. The service resource determines what task definition and revision that Amazon ECS launches. It also determines how many copies of the task definition are launched and what resources are connected to the launched tasks. These connected resources include load balancers and service discovery. The service resource also defines rules for networking and placement of the tasks on hardware.

Use `awsvpc` network mode and give each service its own security group

We recommend that you use `awsvpc` network mode for tasks on Amazon EC2. This allows each task to have a unique IP address with a service-level security group. Doing so creates per-service security group rules, instead of instance-level security groups that are used in other network modes. Using per-service security group rules, you can, for example, authorize one service to

talk to an Amazon RDS database. Another service with a different security group is denied from opening a connection to that Amazon RDS database.

Turn on Amazon ECS managed tags and tag propagation

After you turn on Amazon ECS managed tags and tag propagation, Amazon ECS can attach and propagate tags on the tasks that the service launches. You can customize these tags and use them to create tag dimensions such as `environment=production` or `team=web` or `application=storefront`. These tags are used in usage and billing reports. If you set up the tags correctly, you can use them to see how many vCPU hours or GB hours that a particular environment, team, or application used. This can help you to estimate the overall cost of your infrastructure along different dimensions.